

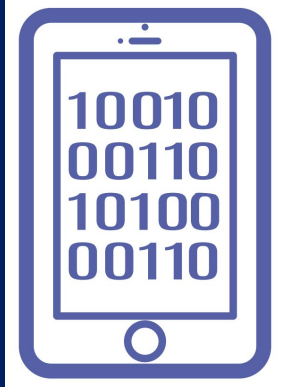
MOBİL UYGULAMA



İSTANBUL DENEYAP TEKNOLOJİ ATÖLYESİ

Ders Saatleri : 09.00 - 13.00 - Lise Sabah Grubu (Sınıfta) - (9 Temmuz- 16 Temmuz Hafta İçi)

Ders Saatleri : 13.30 - 17.30 - Orta Öğle Grubu (Sınıfta) - (9 Temmuz- 16 Temmuz Hafta İçi)



MOBİL UYGULAMA DERSİ

6.Ders: Mobil Uygulamalar-Flutter-Dart



T3 DENEYAP EĞİTMENİ (Hazırlayan ve Sunan): Muhammed Emin Korkunç

(Bilgiyar Mühendisi)

Bu sunumun kopyalanması, paylaşılması ve yazılı izinsiz kullanımı KESİNLİKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunc-100ba2215 | github.com/muhammedkorkunc

MOBİL UYGULAMA

DENEYAP
T Ü R K İ Y E

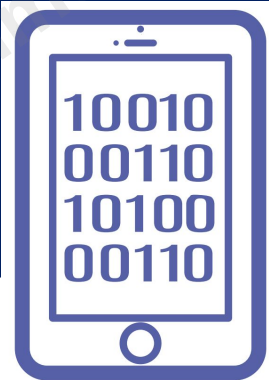
#**MİLLİ**
TEKNOLOJİ
HAMLESİ

Sabah Grubu

- 09.00 - 09.50: 1. Ders
- 09.50 - 10.00: 1. Mola
- 10.00 - 10.50: 2. Ders
- 10.50 - 11.00: 2. Mola
- 11.00 - 11.50: 3. Ders
- 11.50 - 12.00: 3. Mola
- 12.00 - 13.00: 4. Ders

Öğle Grubu

- 13.30 - 14.20: 1. Ders
- 14.20 - 14.30: 1. Mola
- 14.30 - 15.20: 2. Ders
- 15.20 - 15.30: 2. Mola
- 15.30 - 16.20: 3. Ders
- 16.20 - 16.30: 3. Mola
- 16.30 - 17.30: 4. Ders



Bu sunumun kopyalanması, paylaşılması ve yazılı izinsiz kullanımı KESİNLİKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunc-100ba2215 | github.com/muhammedkorkunc

3- TERMINAL

KURULUM VİDEOSU

WINDOWS İÇİN=https://www.youtube.com/watch?v=zdj7Np_gzGw

MACBOOK İÇİN=https://youtu.be/VXswG6ZIZsM?si=B7qt3o5Hlcb_T87y

TERMINAL (flutter_console.bat dosyası içinde)

KOD= flutter doctor –android-licenses

KOD= flutter doctor

KOD= flutter create app

KOD= cd app

KOD= flutter run

 Tek yapmanız gereken, terminalde oluşturduğumuz app1 klasörünü tutup VSCode ekranının üzerine sürükleyip bırakmak!

 Sol taraftaki Explorer (Dosya Gezgini) butonuna tıkladığımızda projenin tüm dalları önümüze serilecek. Bizim için en en önemli dosya hangisi biliyor musunuz? Tabii ki lib klasörünün altındaki main.dart dosyası! Tüm iş bu dosyada dönüyor."

APK YAPIMI

kod : flutter build apk –release

C:\Users\DELL\oyun\build\app\outputs\flutter-apk\app-release.apk

Bu sunumun kopyalanması, paylaşılması ve yazılı izinsiz kullanımı KESİNLİKLE YASAKTIR.

4- 6. Hafta: Form Uygulamaları (Giriş)

Ön Bilgi: Temel Dart bilgisi, temel internet kullanım bilgisi, Flutter ile bileşen üretme bilgisi

Haftanın Kazanımları:

- Form elementleri oluşturabilir
- Form elementlerinden gelen verileri kullanabilir
- Flutter dokümantasyonunu inceleyerek daha önce kullanmadığı bir sınıfı kullanabilir

Haftanın Amacı: Form elementleri içeren bir mobil uygulama oluşturmak.

Kullanılacak Malzemeler: Visual Studio Code, Chrome internet tarayıcısı.

İşleniş: Gözle (Flutter dokümantasyonu) → Uygula (Yapılacaklar Listesi) → Tasarla → Üret → Değerlendir

5- Gözle: Flutter Dokümantasyonu

Açıklama: Flutter, İngilizce hazırlanmış güçlü bir dokümantasyon sistemine sahiptir (docs.flutter.dev). Bir bileşenin sayfası incelendiğinde şu alanlar görülür:

Adım Adım (bir bileşeni araştırmak için):

1. docs.flutter.dev adresine git
2. Sol menüden ya da arama kutusundan bileşenin adını bul (Ör: `SnackBar`)
3. Sayfanın ortasındaki açıklamayı ve kısa video/kod örneklerini incele
4. **Properties** alanından, ihtiyacın olan özelliği (Ör: `backgroundColor`) bul

Flutter **tanımlayıcı (declarative)** bir dil olduğundan, çoğunlukla yalnızca özelliklerin ve yapıcı metotların nasıl kullanılacağını bilmesi yeterlidir.

Alan	İçerik
Classes (sol menü)	Tüm bileşen/sınıf listesi (alfabetik)
Constructors (sağ üst)	Bileşenin yapıcı metotları
Properties (sağ orta)	Bileşenin alabileceği özellikler/parametreler
Methods (sağ alt)	Bileşenin sahip olduğu metotlar (açık renkli olanlar miras alınan metotlardır)

6- Dokümantasyondaki Canlı Kod Editörü ve Cookbook

Canlı Kod Editörü: Her bileşen sayfasının altında, kodun tarayıcıda **çalıştırılıp denenebildiği** gömülü bir editör bulunur (Install SDK / Format / Reset / **Run** düğmeleriyle). Bu editör aslında DartPad'in kendisidir!

Cookbook (Yemek Kitabı): docs.flutter.dev/cookbook adresinde, sık karşılaşılan görevlerin (Ör: bir sayfaya çekmece eklemek, snackbar göstermek, özel font kullanmak) **adım adım tarif** edildiği bir bölümdür. Her tarif kendi başına çalışabilir kod örnekleri içerir.

Adım Adım (bir "tarif" bulmak için):

1. docs.flutter.dev/cookbook adresine git
2. Kategoriyi seç (Ör: **Design, Forms, Navigation**)
3. İlgili tarifi aç (Ör: *Display a snackbar*) → 3 adımlı kodları incele

7- Uygula: SnackBar Gösterimi

Görev: Bir düğme ile gösterilen ve **2 saniye sonra kendiliğinden kaybolan** bir **SnackBar** bileşeni tasarla. ►

DartPad'de tek başına çalışır:

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);
  static const String _title = 'SnackBar Örneği';

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: _title,
      home: Scaffold(
        appBar: AppBar(title: const Text(_title)),
        body: const Center(child: SnackBarDugmesi()),
      ),
    );
  }
}

class SnackBarDugmesi extends StatelessWidget {
  const SnackBarDugmesi({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () {
        final snackBar = SnackBar(
          content: const Text("2 saniye sonra kaybolacağım!"),
          duration: const Duration(seconds: 2),
        );
        ScaffoldMessenger.of(context).showSnackBar(snackBar);
      },
      child: const Text("SnackBar Göster"),
    );
  }
}
```

Kodun Açıklaması: **SnackBar**, ekranın altında kısa süreli mesaj gösteren bir bileşendir.

ScaffoldMessenger.of(context).showSnackBar(snackBar) çağrısı ile ekrana eklenir. **duration** parametresi ne kadar süre görünür kalacağını belirler — süre dolunca otomatik kaybolur.

Çıktı: "SnackBar Göster" yazılı bir düğme görünür. Düğmeye basınca ekranın altından "2 saniye sonra kaybolacağım!" yazılı bir çubuk çıkar ve **2 saniye sonra otomatik kaybolur**.

Açıklama: Kullanıcıdan **form ile veri toplayan** bir *Görev Listesi* (Yapılacaklar Listesi) uygulaması adım adım inşa edilecek.

Uygulamanın 3 hâli:

1. **Boş liste:** "Henüz görev girilmedi" mesajı + soru işareti ikonu
2. **Form ekranı:** Alt taraftan açılan panelde: açıklama kutusu, görev türü seçici (Okul/Ev/İş/Eğlence), tarih seçici, "Ekle" düğmesi
3. **Dolu liste:** Eklenen görev; başlığı, tarihi ve türüne özel bir simgesiyle listede görünür

Kullanılan form elementleri: `TextField` (metin girişi), `DropDownButton` (açılır liste), `showDatePicker` (takvim), `ElevatedButton/TextButton` (düğmeler), `AlertDialog` (hata mesajı)

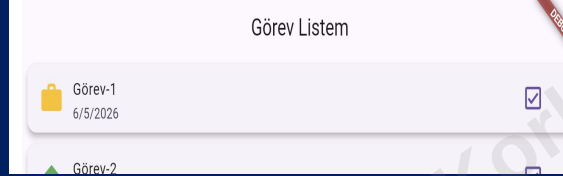
9- Adım Adım: Proje Yapısını Kurmak

Gerçek bir VS Code projesinde izlenecek adımlar:

1. `flutter create gorev_listesi` ile yeni proje oluştur
2. `lib/` klasörü altında iki alt klasör aç: `modals/` (veri modelleri için) ve `widgets/` (bileşenler için)
3. `lib/modals/gorevmodeli.dart` dosyasını oluştur → veri modelini burada tanımla
4. `lib/widgets/gorevlistesi.dart` ve `lib/widgets/yenigorev.dart` dosyalarını oluştur
5. `lib/main.dart` içinde tüm bu dosyaları `import` ederek bir araya getir

DartPad'de çalışırken: DartPad tek bir `main.dart` dosyası kullanır; bu nedenle bu derste tüm sınıflar **tek dosyada** birleştirilerek gösterilecektir (Slayt 21'deki tam kod). Gerçek bir projede ise yukarıdaki gibi dosyalara bölünmesi **daha düzenli ve sürdürülebilir** bir yaklaşımdır.

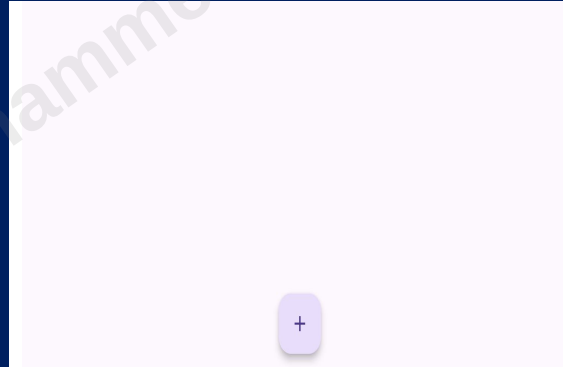
► DartPad'de tek başına çalışır — TÜM UYGULAMA:



Kodun Genel Açıklaması: Bu tek dosyalık uygulama, 4 parçadan oluşur:

1. **GorevModeli** → her görevin veri yapısı
2. **Gorevler** → ana ekran; görev listesini tutar, ekleme/silme metodlarını barındırır, + düğmesiyle formu açar
3. **GorevListesi** → görevleri kartlar hâlinde listeler, her kartta bir

<https://sharetext.io/l9mcs209>



1. yazısı görünür
2. Sağ alttaki + düğmesine basılır → alttan form paneli açılır
3. "Açıklama giriniz" kutusuna bir metin yazılır, sağdaki listeden bir tür seçilir (Ör: Ev)
4. "Tarih Seç" düğmesiyle bir tarih seçilir
5. "Ekle" düğmesine basılır → form kapanır, **yeni görev listede kart olarak görünür** (yeşil ev ikonu + başlık + tarih)
6. Başlık boş bırakılıp "Ekle"ye basılırsa **"Lütfen başlık bilgisi giriniz."** hata penceresi çıkar
7. Bir kartın sağındaki onay kutusuna basılırsa o görev **listeden silinir**

11- GorevModeli Sınıfı (Veri Modeli)

```
class GorevModeli {  
  num id;  
  String baslik;  
  DateTime tarih;  
  Icon simge;  
  GorevModeli({  
    required this.id,  
    required this.baslik,  
    required this.tarih,  
    required this.simge,  
  });  
}
```

Kodun Açıklaması: Her görevin bir kimlik numarası (**id**), bir başlığı (**baslik**), gerçekleştirileceği tarih (**tarih**) ve türünü gösteren bir simgesi (**simge**) vardır. Yapıcı metotta **required** ile **tüm bilgilerin girilmesi zorunlu** kılınmıştır — eksik bilgiyle görev oluşturulamaz.

► DartPad'de tek başına çalışır (modeli test etmek için):

```
import 'package:flutter/material.dart';  
  
class GorevModeli {  
  num id;  
  String baslik;  
  DateTime tarih;  
  Icon simge;  
  GorevModeli({  
    required this.id,  
    required this.baslik,  
    required this.tarih,  
    required this.simge,  
  });  
}  
  
void main() => runApp(const MyApp());  
  
class MyApp extends StatelessWidget {  
  const MyApp({Key? key}) : super(key: key);  
  
  @override
```

Çıktı: Ekranda pembe bir pasta ikonu, "Erdem'in doğum günü" başlığı ve altında "Görev No: 1 — Tarih: 20/7/2026" yazısı görünür. Bu, **GorevModeli** sınıfının doğru çalıştığını kanıtlar.

12- main.dart: Uygulamanın Giriş Kodları

```
import 'package:flutter/material.dart';
import './models/gorevmodeli.dart';
import './widgets/gorevlistesi.dart';
import './widgets/yenigorev.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Görev Listem',
      theme: ThemeData(
        primarySwatch: Colors.purple,
      ),
      home: const Gorevler(),
    );
  }
}
```

Kodun Açıklaması: Uygulamanın başında 3 dosya projeye eklenir: veri modeli (**gorevmodeli.dart**), yeni görev formu (**yenigorev.dart**) ve görev listesi (**gorevlistesi.dart**). **MaterialApp**'in **theme** parametresiyle uygulamanın rengi **mor (purple)** olarak belirlenmiş; **home** parametresine ise ana ekran olan **Gorevler** bileşeni atanmıştır.

13- Gorevler Bileşeni (StatefulWidget İskeleti)

```
class Gorevler extends StatefulWidget {  
  const Gorevler({Key? key}) : super(key: key);  
  @override  
  _GorevlerState createState() => _GorevlerState();  
}  
  
class _GorevlerState extends State<Gorevler> {  
  List<GorevModeli> _gorevler = [];  
  var _gorevSayisi = 0;  
  
  void _gorevEkle(/* ... */) { /* Slayt 10 */ }  
  void _gorevSil(num _gorevId) { /* Slayt 10 */ }  
  void _yeniGorevPenceresi(BuildContext ctx) { /* Slayt  
11 */ }  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text("Görev Listem")),  
      body: GorevListesi(_gorevler, _gorevSil),  
      floatingActionButtonLocation:  
        FloatingActionButtonLocation.centerFloat,  
      floatingActionButton: FloatingActionButton(  
        child: const Icon(Icons.add),  
        onPressed: () => _yeniGorevPenceresi(context),  
      ),  
    );  
  }  
}
```

Kodun Açıklaması: **Gorevler** bileşeni **içeriği kendini güncelleyeceğinden StatefulWidget** tipindedir. Ekrandaki tüm görevlerin listesi (**_gorevler**) ve toplam görev sayısı (**_gorevSayisi**, yeni görevlere **id** vermek için kullanılır) burada tutulur. **build()** metodu bir **Scaffold** oluşturur: gövdesinde **GorevListesi** bileşeni, altında ise ortalanmış bir **+ düğmesi (FloatingActionButton)** vardır. Bu düğmeye basıldığında **_yeniGorevPenceresi** metodu çağrılarak form paneli açılır.

14- _gorevEkle ve _gorevSil Metodları

```
void _gorevEkle(  
    {required String baslik, required  
    DateTime tarih, required Icon simge})  
{  
    setState(() {  
        _gorevler.add(GorevModeli(  
            id: _gorevSayisi,  
            baslik: baslik,  
            tarih: tarih,  
            simge: simge,  
        ));  
    });  
    _gorevSayisi++;  
}
```

```
void _gorevSil(num _gorevId) {  
    setState(() {  
        _gorevler.removeWhere((_gorev)  
=> _gorev.id == _gorevId);  
    });  
}
```

Kodun Açıklaması:

- **_gorevEkle** → **baslik**, **tarih**, **simge** bilgileriyle (hepsi zorunlu) yeni bir **GorevModeli** üretilip **_gorevler** listesine ekler. Bu ekleme **setState()** içinde yapıldığından arayüz otomatik güncellenir. Ardından **_gorevSayisi** bir artırılır (bir sonraki görevin **id**'si için).
- **_gorevSil** → verilen **id**'ye sahip görevi listeden **removeWhere** ile çıkarır; bu işlem de **setState()** içinde olduğundan liste ekranda anında güncellenir.

```
void _yeniGorevPenceresi(BuildContext  
ctx) {  
  showModalBottomSheet(  
    context: ctx,  
    builder: (BuildContext ctx) {  
      return Container(  
        height: 250,  
        child: YeniGorev(_gorevEkle),  
      );  
    },  
  );  
}
```

Kodun Açıklaması:

`showModalBottomSheet`, ekranın altından yukarı doğru açılan ve arka planı karartarak yalnızca panel ile etkileşime izin veren bir bileşen gösterir. `builder` içinde 250 piksel yükseklikte bir `Container` oluşturulup içine `YeniGorev` formu yerleştirilir. `YeniGorev`'e, yeni görev oluşturulduğunda çağrılacak `_gorevEkle` metodu **parametre olarak** gönderilir — bu sayede form kendi içinde `_gorevler` listesine doğrudan erişmeden, üst bileşendeki metodu tetikleyerek veri ekleyebilir.

16- GorevListesi Bileşeni (StatelessWidget)

```
class GorevListesi extends StatelessWidget {  
  List<GorevModeli> _gorevlerim;  
  Function _gorevSil;  
  GorevListesi(this._gorevlerim,  
    this._gorevSil);  
  
  @override  
  Widget build(BuildContext context) {  
    return _gorevlerim.isEmpty  
      ? const ListTile(  
        title: Text("Henüz görev girilmedi"),  
        leading: Icon(Icons.question_mark),  
      )  
      : ListView.builder(  
        /* devamı Slayt 13'te */  
      );  
  }  
}
```

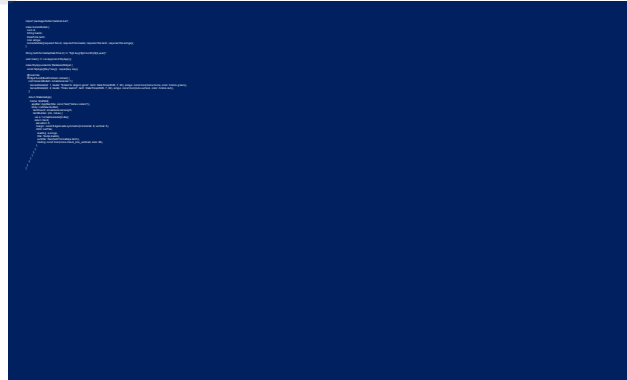
Kodun Açıklaması: **GorevListesi** bileşeni kendi içinde bir durum güncellemesi yapmadığından **StatelessWidget**'tir (güncelleme, üst bileşen olan **Gorevler** içinde yapılır). Yapıcı metodu, görev listesini (**_gorevlerim**) ve silme fonksiyonunu (**_gorevSil**) parametre olarak alır. **build()** içinde **durumsal atama işleci (?)** kullanılarak liste boşsa bir bilgi mesajı (**ListTile**), doluyorsa **ListView.builder** gösterilir.

17- ListView.builder ile Görevleri Listelemek (Card + ListTile)

```
: ListView.builder(  
  itemCount: _gorevlerim.length,  
  itemBuilder: (ctx, index) {  
    GorevModeli e = _gorevlerim[index];  
    return Card(  
      elevation: 5,  
      margin: const EdgeInsets.symmetric(horizontal:  
6, vertical: 5),  
      child: ListTile(  
        leading: e.simge,  
        title: Text(e.baslik),  
        subtitle: Text(tarihFormatla(e.tarih)),  
        trailing: TextButton(  
          child: const Icon(Icons.check_box_outlined,  
size: 28),  
          onPressed: () => _gorevSil(e.id),  
        ),  
      ),  
    );  
  },  
);
```

Kodun Açıklaması: `itemCount`, `_gorevlerim` listesinin uzunluğu kadar satır üretileceğini belirtir. Her satır için `index` değişkeniyle ilgili görev nesnesi (`e`) alınır. Her görev, **kenarları yumuşatılmış ve gölgeli** bir `Card` bileşeni içinde; `ListTile` ile görevin simgesi (`leading`), başlığı (`title`) ve tarihi (`subtitle`) gösterilir. Sağdaki (`trailing`) onay kutusu ikonuna basıldığında `_gorevSil(e.id)` çağrılarak o görev silinir.

► **DartPad'de tek başına çalışır (örnek sabit veriyle):**



Çıktı: Ekranda gölgeli iki kart görünür — biri yeşil ev ikonlu "Erdem'in doğum günü / 20/7/2026", diğeri kırmızı okul ikonlu "Ödev teslimi / 22/7/2026" satırları içerir.

18- YeniGorev Bileşeni (StatefulWidget İskeleti)

```
class YeniGorev extends StatefulWidget {  
  final Function _gorevEkleyici;  
  const YeniGorev(this._gorevEkleyici, {Key? key}) :  
    super(key: key);  
  @override  
  State<YeniGorev> createState() =>  
    _YeniGorevState();  
  _YeniGorevState();  
}  
  
class _YeniGorevState extends State<YeniGorev> {  
  DateTime? _tarih = DateTime.now();  
  String? _baslik;  
  String? _simge = "Okul";  
  final _baslikKontrolcusu = TextEditingController();  
  
  void _tarihSec() { /* Slayt 17 */ }  
  void _gorevUret() { /* Slayt 19 */ }  
  void _hataBildir(String mesaj) { /* Slayt 20 */ }  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      child: Column(/* Slayt 15-18 */),  
    );  
  }  
}
```

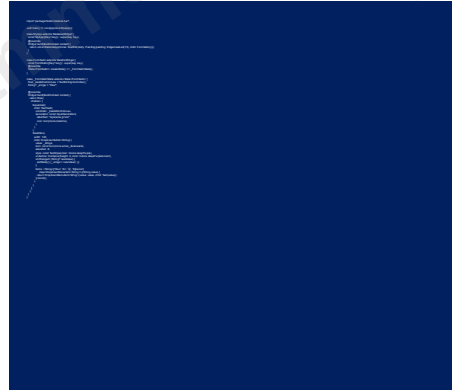
Kodun Açıklaması: **YeniGorev** formunun her alanı (**_tarih**, **_baslik**, **_simge**) değişebildiğinden bu bileşen **StatefulWidget**'tır. **_gorevEkleyici**, üst bileşenden (**Gorevler**) gelen ve yeni görevi listeye ekleyen fonksiyondur. **_baslikKontrolcusu** (**TextEditingController**), metin kutusundaki değere kod içinden ulaşmayı sağlar.

19- Form Alanı 1: Açıklama Kutusu (TextField) ve Görev Türü (DropDownButton)

```
Row(
  children: [
    Expanded(
      child: TextField(
        controller: _baslikKontrolcusu,
        decoration: const InputDecoration(
          labelText: "Açıklama giriniz",
          icon: Icon(Icons.textsms),
        ),
      ),
    ),
    SizedBox(
      width: 100,
      child: DropDownButton<String>(
        value: _simge,
        icon: const Icon(Icons.arrow_downward),
        elevation: 8,
        style: const TextStyle(color: Colors.deepPurple),
        underline: Container(height: 2, color:
Colors.deepPurpleAccent),
        onChanged: (String? newValue) {
          setState(() { _simge = newValue!; });
        },
        items: <String>['Okul', 'Ev', 'İş', 'Eğlence']
          .map<DropDownMenuItem<String>>((String value) {
            return DropDownMenuItem<String>(value: value, child:
Text(value));
          }).toList(),
      ),
    ),
  ],
)
```

Kodun Açıklaması: `TextField` ve `DropDownButton` bileşenlerinin **yan yana** gösterilmesi için bir `Row` içine alınmıştır. `TextField`, kalan tüm genişliği kaplaması için `Expanded` içine; `DropDownButton` ise sabit 100px genişlik için `SizedBox` içine yerleştirilmiştir. **controller:** `_baslikKontrolcusu` sayesinde kutuya yazılan metne kod içinden ulaşılabilir. `DropDownButton`'ın `items` listesi, `String` listesinin `map()` metoduyla `DropDownMenuItem` listesine çevrilerek oluşturulur. `onChanged` ile kullanıcı yeni bir tür seçtiğinde `_simge` değişkeni `setState()` içinde güncellenir.

► DartPad'de tek başına çalışır:



Çıktı: Solda "Açıklama giriniz" etiketli, sol tarafında bir konuşma balonu simgesi olan bir metin kutusu; sağda ise "Okul" yazan, açılınca "Okul / Ev / İş / Eğlence" seçeneklerini gösteren mor renkli bir açılır liste görünür.

```
Row(  
  children: [  
    Expanded(  
      child: TextField(  
        enabled: false,  
        decoration: InputDecoration(  
          labelText: (_tarih == null) ? "Tarih  
seçiniz" : tarihFormatla(_tarih!),  
          icon: const Icon(Icons.date_range),  
        ),  
      ),  
    ),  
    SizedBox(  
      width: 100,  
      child: TextButton(  
        onPressed: _tarihSec,  
        child: const Text("Tarih Seç"),  
      ),  
    ),  
  ],  
)
```

Kodun Açıklaması: Tarih verisi kullanılan dile göre farklı yazılabildiğinden, kullanıcının doğrudan `TextField`'a yazmasına **izin verilmez** (`enabled: false`). Bunun yerine kutunun etiketinde (`labelText`) seçilen tarih (yoksa "Tarih seçiniz" yazısı) gösterilir. Sağdaki, kenarlıksız/artalansız bir `TextButton` ("Tarih Seç") ise `_tarihSec` metodunu tetikleyerek takvim penceresini açar.

21- _tarihSec Metodu: showDatePicker ile Takvim Açmak

```
void _tarihSec() {  
  showDatePicker(  
    context: context,  
    initialDate: DateTime.now(),  
    firstDate:  
    DateTime(DateTime.now().year,  
    lastDate:  
    DateTime(DateTime.now().year +  
3),  
  ).then((pickedDate) {  
    if (pickedDate == null) {  
      return;  
    }  
    setState(() {  
      _tarih = pickedDate;  
    });  
  });  
}
```

Kodun Açıklaması: `showDatePicker`, yerleşik bir takvim diyalogu açar. `initialDate` başlangıçta seçili görünecek tarihi, `firstDate/lastDate` ise seçilebilecek tarih aralığını belirler (burada: bugünden 3 yıl sonrasına kadar). Kullanıcı bir tarih seçtiğinde bu değer `.then()` içindeki `pickedDate` parametresine gelir. Kullanıcı **İptal**'e basarsa `pickedDate null` olur — bu durumda hiçbir işlem yapılmaz (`return`). Tarih seçildiyse `setState()` içinde `_tarih` güncellenir.

► DartPad'de tek başına çalışır:

```
import 'package:flutter/material.dart';  
  
String tarihFormatla(DateTime d) => "${d.day}/${d.month}/${d.year}";  
  
void main() => runApp(const MyApp());  
  
class MyApp extends StatelessWidget {  
  const MyApp({Key? key}) : super(key: key);  
  @override  
  Widget build(BuildContext context) {  
    return const MaterialApp(home: Scaffold(body: Center(child: TarihSecici())));  
  }  
}  
  
class TarihSecici extends StatefulWidget {  
  const TarihSecici({Key? key}) : super(key: key);  
  @override  
  State<TarihSecici> createState() => _TarihSeciciState();  
}  
  
class _TarihSeciciState extends State<TarihSecici> {  
  DateTime? _tarih;  
  
  void _tarihSec() {  
    showDatePicker(  
      context: context,  
      initialDate: DateTime.now(),
```

Çıktı: "Tarih seçilmedi" yazısı ve "Tarih Seç" düğmesi görünür. Düğmeye basılınca bir takvim açılır; bir gün seçilince ekrandaki yazı "Seçilen tarih: GG/AA/YYYY" şeklinde güncellenir.

```
Padding(  
  padding: const  
EdgeInsets.fromLTRB(0, 20, 0, 20),  
  child: ElevatedButton(  
    onPressed: _gorevUret,  
    child: const Text('Ekle'),  
  ),  
)
```

Kodun Açıklaması: Formun en altında yer alan bu **ElevatedButton**, dokunulduğunda **_gorevUret** metodunu çağırır. Üstten ve alttan 20 piksellik boşluk bırakmak için bir **Padding** içine alınmıştır.

23- _gorevUret Metodu: Formdaki Verilerden Görev Üretmek

```
void _gorevUret() {  
    if (_baslikKontrolcusu.text.isEmpty) {  
        _hataBildir("Lütfen başlık bilgisi giriniz.");  
        return;  
    } else {  
        _baslik = _baslikKontrolcusu.text;  
    }  
  
    Icon? simgem;  
    switch (_simge) {  
        case "Okul":  
            simgem = const Icon(Icons.school, color: Colors.red, size:  
32);  
            break;  
        case "Ev":  
            simgem = const Icon(Icons.home, color: Colors.green,  
size: 32);  
            break;  
        case "İş":  
            simgem = const Icon(Icons.work, color: Colors.amber, size:  
32);  
            break;  
        default:  
            simgem = const Icon(Icons.videogame_asset, color:  
Colors.blueAccent, size: 32);  
    }  
  
    widget._gorevEkleyici(baslik: _baslik!, tarih: _tarih!, simge:  
simgem!);  
    Navigator.of(context).pop();  
}
```

Kodun Açıklaması:

1. Önce başlık kutusu boş mu diye kontrol edilir; boşsa **_hataBildir** ile hata gösterilip metot sonlandırılır (**return**)
2. Doluysa, kutudaki metin **_baslik** değişkenine aktarılır
3. **switch-case** ile, **DropDownButton**'dan gelen **_simge** değerine göre uygun **ikon, renk ve boyut** belirlenir (**Okul** → kırmızı okul ikonu, **Ev** → yeşil ev ikonu, **İş** → sarı iş çantası ikonu, diğerleri/**Eğlence** → mavi oyun ikonu)
4. **widget._gorevEkleyici(...)** çağrısıyla, üst bileşendeki **_gorevEkle** metoduna veriler gönderilir — **widget.** öneki, **StatefulWidget**'ın kendisine (**YeniGorev**) ait değişkenlere **State** sınıfı içinden erişmek için kullanılır
5. Son olarak **Navigator.of(context).pop()** ile form paneli kapatılır

24- _hataBildir Metodu: AlertDialog ile Hata Göstermek ve Son

```
void _hataBildir(String mesaj) {
  final hataMesaji = AlertDialog(
    title: const Text('Hata'),
    content: Text(mesaj),
    actions: <Widget>[
      TextButton(
        onPressed: () =>
          Navigator.pop(context, 'Tamam'),
        child: const Text('Tamam'),
      ),
    ],
  );
  showDialog(
    context: context,
    builder: (BuildContext context) {
      return hataMesaji;
    },
  );
}
```

Kodun Açıklaması: **AlertDialog**, ekranın ortasında beliren, kullanıcının **dikkatini toplaması gereken** kısa mesajlar için kullanılan bir bileşendir. **title** başlığı, **content** asıl mesajı, **actions** ise altta gösterilecek düğmeleri (burada tek bir "Tamam" düğmesi) tutar. **showDialog** metodu bu diyalogu ekrana getirir; "Tamam" düğmesine basıldığında **Navigator.pop()** ile diyalog kapatılır.

► DartPad'de tek başına çalışır:

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(home: Scaffold(body: Center(child: HataDugmesi())));
  }
}

class HataDugmesi extends StatelessWidget {
  const HataDugmesi({Key? key}) : super(key: key);

  void _hataBildir(BuildContext context, String mesaj) {
    final hataMesaji = AlertDialog(
      title: const Text('Hata'),
      content: Text(mesaj),
      actions: <Widget>[
        TextButton(
          onPressed: () => Navigator.pop(context, 'Tamam'),
          child: const Text('Tamam'),
        ),
      ],
    );
    showDialog(context: context, builder: (BuildContext context) => hataMesaji);
  }
}
```

Çıktı: "Hata Göster" düğmesine basılınca ekranın ortasında beyaz bir kutu belirir; üstte "Hata" başlığı, altında "Lütfen başlık bilgisi giriniz." mesajı ve sağ altta "Tamam" yazısı görünür. "Tamam"a basılınca diyalog kapanır.

25- Tasarla ve Üret Görevi

Görev: Basit bir **iletişim listesi** uygulaması üret. Bu uygulamada kişilerin:

- Ad Soyad
- Telefon numarası
- İşyeri / okul bilgisi
- E-posta
- Not ortalaması
- Doğum tarihi

bilgilerini alan bir **form** kullan (bugünkü **YeniGorev** formuna çok benzer bir yapı kurulabilir — **TextField**'lar, gerekirse **keyboardType**, ve **showDatePicker** ile doğum tarihi).

Adım Adım öneri:

1. **KisiModeli** adında (Ad, telefon, işyeri, e-posta, not ortalaması, doğum tarihi alanlı) bir veri modeli sınıfı yaz
2. **YeniGorev** formuna benzer bir **YeniKisi** formu oluştur; her alan için ayrı **TextField** kullan
3. Telefon ve not ortalaması alanlarında **keyboardType** ile uygun klavyeyi göster (bir sonraki slaytta)
4. Doğum tarihi için **showDatePicker** kullan
5. Girilen kişileri **ListView.builder** + **Card/ListTile** ile listele

26- Değerlendir + İlave Etkinlik: keyboardType Parametresi

Değerlendirme: Öğrencilerin eğitim sonunda programlamaya yönelik tutumlarını ölçmek için öz değerlendirme formu uygulanır (form bağlantıları kitapta verilmiştir).

İlave Etkinlik — keyboardType Parametresi: `TextField`'in `keyboardType` parametresi, mobil cihazda gösterilecek **sanal klavyenin türünü** değiştirir. Bu sayede alınacak bilgi türüne göre (Ör: sayısal veri, e-posta) daha uygun bir klavye gösterilebilir.

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text("keyboardType Örneği")),
        body: const Padding(
          padding: EdgeInsets.all(16),
          child: Column(
            children: [
              TextField(
                decoration: InputDecoration(labelText: "Normal metin klavyesi"),
                keyboardType: TextInputType.text,
              ),
              SizedBox(height: 16),
              TextField(
                decoration: InputDecoration(labelText: "Sayısal klavye (telefon)"),
                keyboardType: TextInputType.number,
              ),
              SizedBox(height: 16),
              TextField(
                decoration: InputDecoration(labelText: "E-posta klavyesi"),
                keyboardType: TextInputType.emailAddress,
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Kodun Açıklaması: `TextInputType.text` (varsayılan) normal harf klavyesini, `TextInputType.number` yalnızca rakamlardan oluşan sayısal klavyeyi, `TextInputType.emailAddress` ise @ işaretini kolayca girmeye yarayan e-posta klavyesini açar.

Çıktı (özellikle mobil cihaz/emülatörde belirgindir): Her metin kutusuna dokunulduğunda **farklı bir sanal klavye** açılır — ikinci kutuda yalnızca rakam tuşları, üçüncü kutuda ise @ kısayolu bulunan bir klavye görünür.

Bu hafta öğrendiklerimiz:

1. Flutter'ın resmi dokümantasyonunu ve Cookbook'unu **kendi başına** kullanabilme
2. `SnackBar` ile kısa süreli bilgilendirme mesajları gösterme
3. `TextField` + `TextEditingController` ile metin girişi alma
4. `DropDownButton` ile seçenek listesi sunma
5. `showDatePicker` ile tarih seçtirme
6. `showModalBottomSheet` ile alttan açılan form paneli oluşturma
7. `AlertDialog` + `showDialog` ile hata/uyarı mesajı gösterme
8. Formdan toplanan verilerle bir veri modeli (`GorevModeli`) nesnesi üretip listeye ekleme

28- PROJE DAĞITIMI

 Kategori 1: Oyun Projeleri (1 - 10)



Kategori 2: Günlük Yaşam ve Alışkanlık Araçları (11 - 20)



Kategori 3: Alışveriş, E-Ticaret ve Finans (21 - 30)



Kategori 4: Eğitim, Zeka ve Hesaplama (31 - 40)



Kategori 5: Dini, Toplumsal ve Kültürel Uygulamalar (41 - 50)



Kategori 6: İngilizce Öğrenme Uygulamaları (51 - 60)



Kategori 7: Sağlık ve Sağlıklı Yaşam Uygulamaları (61 - 70)

PROJE KONULARI

https://docs.google.com/document/d/1ekC6ir-T466zD2b_YchbTJzyHepVix34adO9FnWtztc/edit?hl=tr&tab=t.0#heading=h.hsjzfigkr0zz